Sprint 14 go-live pack: the hosted demo

Your accounts, your keys. Every step here happens in your own Supabase, Vercel, Resend, Anthropic and Voyage accounts. No key, password or token goes into this file, the chat or the repository. Where a step needs one, it says where you keep it.

Branch `main` (it holds `vercel.json`, the hosted build checks, the demo data, the daily ping and `supabase/functions/.env.hosted.example`). About 90 minutes, most of it waiting for deployments. Do the steps in order: later ones need values from earlier ones.

Two values you will note as you go (they are not secret): - **Project ref**: the Supabase project's ID (in its address, `https://<project-ref>.supabase.co`). - **App address**: the Vercel address, for example `https://rfi-respond.vercel.app`.

Before you start

- A **Resend** account (or another SMTP provider) with a sending domain you own verified. Resend shows the DNS records to add; verification can take a while, so start it first.
- **Two new keys for the demo**, one from Anthropic and one from Voyage, each in its own workspace or project with a monthly spend limit. Not the keys you use locally, so the demo's spend is capped on its own and its key can be revoked alone.
- **Two-factor sign-in** on every account involved: Supabase (and, under the organisation's settings, require it for all members), Vercel, GitHub, Resend, Anthropic and Voyage (which already has payment details, so its rate limit will not stall processing). Anyone who gets into one of these gets the demo, whatever the app does.
- Docker Desktop is not needed for these steps.

Steps

1. Create the Supabase project

Flow

1. In the Supabase dashboard, choose **New project** in your Free organisation (with Campaign Studio paused, it has a free slot): name `rfi-respond-demo`, region **London (eu-west-2)**.
2. Generate a database password and save it in your password manager.
3. Straight away, before anything is public (security review, M4), under **Authentication**:
    - **Sign In / Providers**: turn **Allow new users to sign up** off. Keep **Email** on, with **Confirm email** on.
    - Set **Password requirements**: minimum length `12`, **Letters and digits**. Leaked-password protection is greyed out on the Free plan (agreed, Q1).
    - **Rate Limits**: set emails sent per hour to `10`. This limits how much mail anyone can make the project send, for example password-reset requests for the demo's fictional addresses.

Expected outcome: the project is ready, its address is `https://<project-ref>.supabase.co`, and sign-up is off.

Result: ☐ Done. Project ref:

2. Apply the migrations and deploy the functions

Flow (PowerShell in `C:\Code\rfi-responder`, on `main`)

1. `npx supabase login`
2. `npx supabase link --project-ref <project-ref>` and enter the database password when asked.
3. `npx supabase db push`, then answer `Y`.
4. `npx supabase functions deploy`

Expected outcome

- `db push` lists the migrations from `20260430145610` (the prototype's) to `20261102110000` and applies them all without an error.
- `functions deploy` deploys `embed`, `embed-entries`, `extract-questions` and `process-questions`.

Result: ☐ Done. Notes:

3. Set the function secrets

Flow

1. Copy `supabase\functions\.env.hosted.example` to `supabase\functions\.env.hosted.local`, and put the two demo keys in it. Git ignores `*.local` files.
2. `npx supabase secrets set --env-file supabase/functions/.env.hosted.local`. This reads the keys from the file, so they never appear on the command line or in your shell history. It sets only the four values in that file, not everything in your local `.env`.
3. `npx supabase secrets list`

Expected outcome: the list shows `ANTHROPIC_API_KEY`, `VOYAGE_API_KEY`, `EMBEDDING_PROVIDER` and `EMBEDDING_MODEL`, with their values hidden, and nothing else of yours. `APP_ORIGIN` comes in step 5.
Result: ☐ Done. Notes:

4. Deploy the app on Vercel

Flow

1. In Supabase, open **Project Settings**, then **API**, and copy the **Project URL** and the **anon** (or **publishable**) key. Never copy the `service_role` or secret key.
2. In Vercel, choose **Add New**, then **Project**, and import `rorymcclelland/RFI-responder`.
3. Under **Environment Variables**, add `VITE_SUPABASE_URL` (the Project URL) and `VITE_SUPABASE_ANON_KEY` (the anon key), for Production and Preview. Add nothing else: the build refuses a secret key, and stops if a provider key ends up in the built app. Leave the build settings as `vercel.json` sets them.
4. Choose **Deploy**.
5. In the project's **Settings**, then **Deployment Protection**, keep **Vercel Authentication** on for preview deployments, so only you can open builds of other branches.

Expected outcome

- The build log shows `npm run build:hosted` passing, then the deployment is ready.
- Opening the app address shows the RFI Respond sign-in page.
- If the build stops with “The hosted build stopped”, or “The built app contains what must never reach a browser”, the message names what to fix.

Result: ☐ Done. App address:

5. Point Supabase at the app

Flow

1. `npx supabase secrets set APP_ORIGIN=<app address>` (no trailing slash).
2. In Supabase, open **Authentication**, then **URL Configuration**. Set **Site URL** to the app address, and add `<app address>/**` under **Redirect URLs**.

Expected outcome: the secret is listed, and both addresses are saved.
Result: ☐ Done. Notes:

6. Email

Flow

1. **Authentication**, then **Emails**, then **SMTP Settings**: turn on custom SMTP.
    - Host `smtp.resend.com`, port `465`, user `resend`.
    - Password: a Resend API key with send access only (create it in Resend).
    - Sender: an address on your verified domain, with the sender name `RFI Respond demo`.
2. Check **Authentication**, then **Rate Limits** still says `10` emails an hour: turning on custom SMTP can reset it to its default (30).

Expected outcome: all settings saved; Supabase accepts the SMTP settings.
Result: ☐ Done. Notes:

7. Load the demo data

Flow

1. In Supabase, open the **SQL Editor** and start a new query.
2. Paste the whole of `supabase\demo-hosted.sql` and choose **Run**.
3. Run `select count(*) from kb_entries;`, `select name from opportunities;` and `select email from auth.users;`

Expected outcome

- The first run reports success.
- There are 132 entries: 130 seeded and 2 learned by the demo’s exported response.
- There are four responses, each marked “(fictional)”.
- `auth.users` holds only the eight fictional people, all at `@example.invalid`. Any other address means someone signed up before step 1 took effect: delete it under **Authentication**, then **Users**.

Result: ☐ Done. Notes:

8. Your admin account

Flow

1. **Authentication**, then **Users**, then **Invite user**, with your own email address.
2. Open the email, follow the link, and set a password (12 characters or more, letters and digits). Save it in your password manager.
3. In the SQL Editor, run this, with your address in both places. It makes you the first admin and records that in the audit log:

```
update public.user_profiles set is_admin = true
 where id = (select id from auth.users where lower(email) =
lower('you@yourdomain'))
 returning id, display_name, is_admin;
insert into public.audit_log (actor, action, target_id, details)
select id, 'user.admin_granted', id, '{"by": "go-live step 8 (first
admin)"}'::jsonb
  from auth.users where lower(email) = lower('you@yourdomain');
```

4. Sign out of the app and back in, and wait a minute on the overview. The first sign-in indexes the 132 entries with Voyage (a few pence, well inside the daily limit).

Expected outcome

- The email arrives from your sending domain.
- The link opens the app's **Welcome: choose a password** page.
- The update returns one row, with `is_admin` true. If it returns none, the address does not match the invitation's.
- After signing in again you see **Governance** in the menu, and no indexing error appears.

Result: ☐ Done. Notes:

9. Limits, in the app and at the providers

Flow

1. In the app, open **Governance**, then **Usage and limits**. Set:
   - **Monthly budget (£)** to `5`;
   - **Questions per person per day** to `50`;
   - **Word documents read by AI per person per day** to `5`.
2. In the Anthropic console, set a monthly spend limit for the key's workspace.
3. In the Voyage dashboard, set a usage limit, and switch on the zero-retention opt-out (`Organization`, then `Terms of Service`, the dashboard's labels).

Expected outcome: the governance page shows the new limits. Both provider limits are saved, and the Voyage opt-out is on.
Result: ☐ Done. Notes:

10. Invite the reviewers

Flow

1. For each reviewer: **Authentication**, then **Users**, then **Invite user**, with their address.
2. Tell them to expect the email, and that the demo holds synthetic data only.

Expected outcome: each appears under **Users** as invited; they are members, not admins. `select email from auth.users;` lists only you, your reviewers and the eight fictional people.
Result: ☐ Done. Notes:

11. The daily ping

Flow

1. In GitHub, open the repository's **Settings**, then **Secrets and variables**, then **Actions**.
2. Under **Variables**, add `DEMO_SUPABASE_URL` with the Project URL.
3. Under **Secrets**, add `DEMO_SUPABASE_ANON_KEY` with the anon (publishable) key, the same one you gave Vercel. Never the service-role or secret key.
4. Open **Actions**, choose **Keep the demo awake**, then **Run workflow**.

Expected outcome: the run passes and its log ends "The demo database answered." From then on it runs four times a day.
Result: ☐ Done. Notes:

If the demo is paused anyway

The ping is meant to keep the project active: Supabase says "typically a few user requests to the database each day" are enough, but does not say what it counts, so about eight days after go-live check in the dashboard that the project is still active, and tell me if it was paused. If the ping ever fails (its run turns red) or GitHub disables it (scheduled workflows in a public repository stop after 60 days without repository activity), Supabase may pause the project after a week. Open the project in the dashboard and choose **Restore project**, wait until it is ready, load the app once, and re-enable the workflow under **Actions** if needed.

To reset the demo

Paste the whole of `supabase\demo-hosted-reset.sql` into the SQL Editor and run it. It removes every response and knowledge-base entry, including what reviewers added, and loads the demo again. Accounts, roles, limits, real people's usage and the audit log are kept; the demo's own recorded usage is replaced, not added to. The next sign-in indexes the entries again (a few pence).

When you have finished

Reply "live", with the app address. I then check the hosted project's security advisors and the app's pages (with your permission, through the connected Supabase tools and the browser), and send the hosted test pack.